Localhost and ports

Objectives

- Explain what localhost (127.0.0.1) refers to
- Read the host and port in a URL such as http://localhost:8080
- Recognise "connection refused" and "address already in use" and what each means

Teaching

Start with a guess

Look at the programs on your laptop, then guess. Practice prediction, not scored.

localhost and ports

- localhost is a name for this same computer: IPv4 127.0.0.1, IPv6 ::1. A request to it never leaves your machine.
- Ports: one computer runs many programs that answer requests; each listens on its own port, a number up to 65535.
- Reading a URL: http://localhost:8080/health has host localhost, port 8080, path /health.
- No port: http uses 80. Nothing listening: connection refused.

One port, one listener

Normally only one program at a time can listen on a port. Starting another on a busy port fails with an error such as address already in use; the exact wording depends on the tool.

Fix it by stopping the old program or by choosing a free port.

Apply it to a new case

Two laptops this time. Ask which computer localhost names.

Model

Programs and ports. A simulation: no real network requests are made.

Your laptop localhost = 127.0.0.1

- 3000 web app, listening
- 5000 nothing listening
- 8080 api, listening

Ben's laptop another computer

- 8080 api, listening

A request to localhost never reaches this machine.

No request sent yet.

Recap

- localhost (127.0.0.1) always means the computer you are on.
- The port picks which program gets the request: http://localhost:8080 means port 8080.
- connection refused: nothing listens there. address already in use: something already does.
- localhost never reaches another computer, such as a teammate's laptop.

Next: back to the course index for the next module

Cheat sheet

localhost, 127.0.0.1	this computer
http://host:port/path	parts of a URL
no port in an http URL	port 80
connection refused	nothing listening on that port
address already in use	port taken: stop the old program or pick another port

Error wording differs between browsers, frameworks and operating systems; what each error means does not.

Sources

Verified against these live sources on 2026-10-06. Commands and messages follow GNU/Linux (bash, coreutils, grep); macOS and other systems can differ.

- RFC 6761 section 6.3: localhost names
- RFC 6890 section 2.2.2: 127.0.0.0/8 loopback

- RFC 9110 section 4.2.1: http URIs and default port 80
- RFC 6335 section 6: port number ranges
- Linux man-pages 6.19: connect(2) ECONNREFUSED
- Linux man-pages 6.19: bind(2) EADDRINUSE

Static interaction alternatives

predict

Prediction (practice, not scored): nothing listens on port 5000 and you open http://localhost:5000. Options: Connection refused (correct: localhost is your own computer, and no program listens on 5000); the API on 8080 answers anyway (a request goes only to the port in the URL); it reaches another computer named localhost (localhost always means this computer); a new program starts on port 5000 (a request never starts a program).

explore

URL builder: choose a host (localhost or 127.0.0.1, which name the same computer) and a port. On this laptop the web app listens on 3000 and the API on 8080. http://localhost:8080/health reaches the API. A port with no listener, such as 5000, or no port, which means the http default 80, gets connection refused.

explore

Send request: sends the request built in the URL builder and shows which program answered or that the connection was refused. Reach the API with port 8080; get connection refused with a port where nothing listens.

explore

Second API activity: start a second copy of the API on a free port. Port 8080 fails with address already in use, because the first API is already listening there. Port 3000 fails for the same reason, because the web app uses it. Port 8081 is free, so the second copy starts.

mcq

Apply question: Ben's API runs on his laptop on port 8080; your laptop runs nothing on 8080; you open http://localhost:8080 and get connection refused. Why? Port 8080 is blocked for everyone (no: the request simply went to your laptop); localhost only works with https (no: plain http works); localhost is your own laptop, where nothing listens on 8080 (correct); Ben must restart his API (no: the request never reached his laptop). To reach Ben's API you need his machine's address.